

MQTT IoT Communication Protocol

MQTT IoT Communication Protocol

1. MQTT Overview
2. Core Concepts
 - 2.1 Publish/Subscribe Model
 - 2.2 Topic
 - 2.3 QoS (Quality of Service)
3. ESP32-S3 MQTT Hardware Features
4. Project Architecture and Flow
 - MQTT Event Callback Flow
5. Hardware Dependencies
6. Source Code Analysis
 - 6.1 Header Files and Macro Configuration
 - 6.2 WiFi Initialization and Event Callback (Review)
 - 6.3 mqtt_event_handler() — MQTT Event Callback
 - 6.4 vMqttTask() — MQTT Main Task
 - 6.5 esp_mqtt_client_publish() Parameter Description
7. Operation Flow
 - 7.1 Build and Flash Flow
 - 7.2 Testing Method
 - 7.3 Normal Log
8. Common Problems

1. MQTT Overview

MQTT (Message Queuing Telemetry Transport) is the most mainstream lightweight publish/subscribe messaging protocol in the IoT field. It is based on TCP and uses **publish/subscribe (Pub/Sub)** model instead of traditional client-server direct connection, achieving **producer-consumer space-time decoupling**.

Importance: MQTT is the de facto standard for IoT communications, natively supported by almost all major cloud platforms including Alibaba Cloud IoT, Huawei Cloud IoT, AWS IoT, and Azure IoT. Its protocol overhead is extremely small (minimum only 2-byte fixed header), especially suitable for bandwidth-constrained, power-sensitive embedded devices.

Application Scenarios:

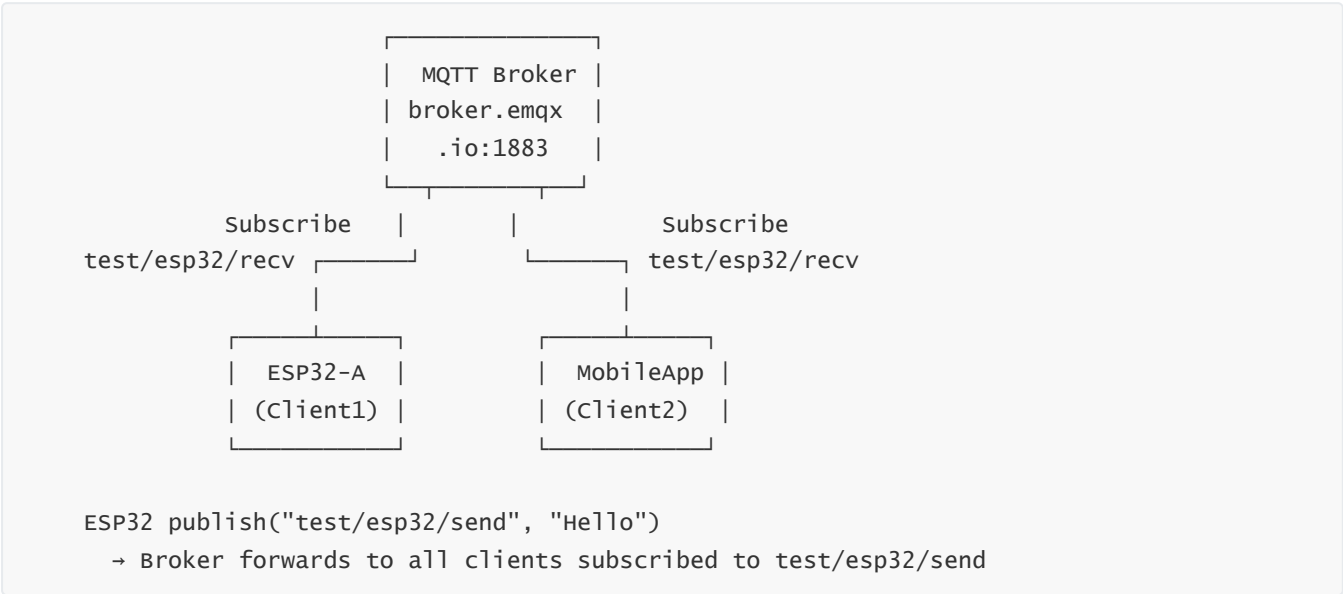
- **Smart Home:** Temperature/humidity sensors report data periodically, mobile app remotely controls lights/AC
- **Industrial IoT:** PLC devices upload production line status in real-time, cloud sends start/stop commands
- **Connected Vehicles:** Vehicle OBD data remote collection, remote lock/unlock control
- **Smart Agriculture:** Soil moisture sensors report regularly, automatic irrigation system triggers based on threshold
- **Smart Energy:** Electricity/water meter remote reading, photovoltaic inverter power generation data real-time monitoring

Feature	MQTT	HTTP
Communication model	Publish/Subscribe (Broker relay)	Request/Response (direct)
Message direction	Bidirectional (publish + subscribe)	Client actively requests
Protocol overhead	Min 2-byte fixed header	Hundreds of bytes of Headers
Real-time push	Native support (subscribe = push)	Need polling or WebSocket
QoS levels	0/1/2 three levels	None
Will message	Supported (auto-publish on disconnect)	Not supported
Typical applications	IoT sensor reporting, smart home	Web API, web browsing

This project implements MQTT client, connecting to public Broker `broker.emqx.io:1883`, subscribing to `test/esp32/recv` topic to receive messages, publishing one message to `test/esp32/send` every 3 seconds.

2. Core Concepts

2.1 Publish/Subscribe Model



2.2 Topic

MQTT routes messages through **topic strings**, supporting hierarchical structures:

Topic	Description
<code>test/esp32/send</code>	ESP32 publishes to this topic
<code>test/esp32/recv</code>	ESP32 subscribes to this topic
<code>sensor/temperature</code>	Temperature sensor data

Topic	Description
<code>home/bedroom/light</code>	Bedroom light control
<code>home/+/light</code>	Wildcard "+" matches single level (e.g. bedroom, kitchen)
<code>home/#</code>	Wildcard "#" matches multiple levels (e.g. bedroom/light/status)

2.3 QoS (Quality of Service)

QoS	Name	Semantics	Applicable Scenario
0	At most once	Fire and forget, no delivery guarantee	High-frequency sensor data (no impact if some frames lost)
1	At least once	Guaranteed delivery, may duplicate	Important control commands
2	Exactly once	Guaranteed delivery and no duplication	Billing/payment information

This project uses QoS 0, suitable for periodic reporting scenarios.

3. ESP32-S3 MQTT Hardware Features

MQTT is based on TCP. After ESP32-S3 connects via WiFi, it uses `espressif__mqtt` component to implement client functionality.

Feature	Description
Transport layer	TCP (default 1883) or TLS/SSL (8883)
Broker	<code>broker.emqx.io:1883</code> (EMQX public test server)
Required component	<code>espressif__mqtt</code> (must declare in <code>idf_component.yml</code>)
Keep Alive	Default 120s, broker considers device offline if no communication beyond this

4. Project Architecture and Flow

```
app_main()
└─ xTaskCreate(vMqttTask, stack=8KB, prio=5)
    └─ vMqttTask():
        │─ wifi_init()                ← WiFi connect + wait for IP
        │
        │─ esp_mqtt_client_init(&mqtt_cfg)    ← Create MQTT client
        │   └─ .broker.address.uri = BROKER_URI
        │
        │─ esp_mqtt_client_register_event()    ← Register event callback
```

```

|   └─ ESP_EVENT_ANY_ID → mqtt_event_handler
|
└─ esp_mqtt_client_start()           ← Start client (async connect)
|
└─ while(1):                         ← Publish loop
    └─ esp_mqtt_client_publish(TOPIC_SEND, "Hello ESP32 MQTT")
        └─ vTaskDelay(3000ms)

```

MQTT Event Callback Flow

```

MQTT_EVENT_CONNECTED           ← Broker connected successfully
→ esp_mqtt_client_subscribe(TOPIC_RECV) → Auto subscribe to topic
→ ESP_LOGI "Subscribed to receive topics"

MQTT_EVENT_DATA                ← Received subscribed topic message
→ Print event->data_len + event->data

```

5. Hardware Dependencies

- **Hardware Connection:** No external wiring, WiFi wireless communication
- **Required Components:** `mqtt_client` (espressif_mqtt), `esp_wifi`, `esp_netif`, `esp_event`, `nvs_flash`

Note: `esp_mqtt` is a third-party component from ESP-IDF Component Registry, not built into IDF. Before using, must declare dependency in `main/idf_component.yml`, otherwise compilation will fail to find the header file:

```

## IDF Component Manager Manifest File
dependencies:
idf:
  version: '>=4.1.0'
espressif/mqtt: "*"

```

After declaration, execute `idf.py build`, the component manager will automatically download the component to `managed_components/` directory.

6. Source Code Analysis

Full source: `Source Code` → `ESP-IDF` → `5.Network_Protocol` → `7.MQTT`

6.1 Header Files and Macro Configuration

First, include required headers, and define WiFi connection parameters, Broker address, and publish/subscribe topics.

```

#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"

```

```

#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "esp_log.h"
#include "nvs_flash.h"
#include "esp_netif.h"
#include "esp_event.h"
#include "esp_wifi.h"
#include "mqtt_client.h"

#define WIFI_SSID      "Your_WiFi_SSID"
#define WIFI_PASS      "Your_WiFi_Password"
#define BROKER_URI     "mqtt://broker.emqx.io:1883"
// Publish topic
#define TOPIC_SEND      "test/esp32/send"
// Subscribe topic
#define TOPIC_RECV      "test/esp32/recv"
#define WIFI_TIMEOUT_MS 15000

```

6.2 WiFi Initialization and Event Callback (Review)

The WiFi STA connection flow in this experiment (NVS init, `wifi_init()`, WiFi event callback `wifi_event_handler()`) was detailed in **5.2 WIFI-STA Mode** and won't be repeated here. After WiFi connects, event group notifies MQTT task to continue.

WiFi event callback details: 5. Network Protocols and Wireless Communication -> 2. WIFI-STA Mode
-> WIFI-STA Mode

6.3 mqtt_event_handler() — MQTT Event Callback

`mqtt_event_handler()` only cares about two moments: connected to Broker, received message. After connecting, first thing is to call `esp_mqtt_client_subscribe()` to subscribe to `test/esp32/recv`—after subscribing, when anyone sends to this Topic, you automatically receive it. QoS 0 means fire and forget, suitable for high-frequency sensor data. When message received, `MQTT_EVENT_DATA` triggers, print message content and length from `event->data`.

```

static void mqtt_event_handler(void *arg, esp_event_base_t base,
                               int32_t event_id, void *event_data)
{
    esp_mqtt_event_handle_t event =
        (esp_mqtt_event_handle_t)event_data;

    switch (event->event_id) {
        case MQTT_EVENT_CONNECTED:
            ESP_LOGI(TAG, "MQTT Connected");
            // Subscribe to topic (QoS 0 = at most once)
            esp_mqtt_client_subscribe(event->client,
                                      TOPIC_RECV, 0);
            ESP_LOGI(TAG, "Subscribed to topic: %s",
                     TOPIC_RECV);
            break;

        case MQTT_EVENT_DATA:
            // Print received data

```

```

        ESP_LOGI(TAG, "Receive: %.*s",
                  event->data_len, event->data);
        break;

    default:
        break;
}
}

```

6.4 vMqttTask() — MQTT Main Task

`vMqttTask()` first connects to WiFi, then to MQTT Broker, then in infinite loop sends one message outward every 3 seconds. `esp_mqtt_client_start()` is asynchronous—it returns immediately, connection process runs in background, after connecting, `MQTT_EVENT_CONNECTED` callback notifies you.

`esp_mqtt_client_publish()` parameters: `topic` specifies which topic to publish to, `data` is message content, `len` fill 0 for SDK to auto-calculate length (`strlen`), `qos` fill 0 means fire and forget without confirmation, `retain` fill 0 means don't let Broker cache this message for later subscribers.

```

void vMqttTask(void *pv)
{
    // Init WiFi
    wifi_init();

    // Set Broker address
    esp_mqtt_client_config_t mqtt_cfg = {
        .broker.address.uri = BROKER_URI,
    };

    // Create MQTT client
    esp_mqtt_client_handle_t client =
        esp_mqtt_client_init(&mqtt_cfg);

    // Register MQTT event callback
    esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID,
                                   mqtt_event_handler, NULL);

    // Start MQTT client (async)
    esp_mqtt_client_start(client);

    // Loop to publish messages
    while (1) {
        char msg[] = "Hello ESP32 MQTT";
        // Params: client, topic, message, length(0=auto),
        //          qos(0), retain(0)
        esp_mqtt_client_publish(client,
                                TOPIC_SEND,
                                msg,
                                0,
                                0,
                                0);

        ESP_LOGI(TAG, "Send message: %s", msg);
    }
}

```

```

        // 3s interval
        vTaskDelay(pdMS_TO_TICKS(3000));
    }
}

```

6.5 esp_mqtt_client_publish() Parameter Description

`esp_mqtt_client_publish()` is the core API for MQTT message publishing. Parameters:

```

int esp_mqtt_client_publish(
    esp_mqtt_client_handle_t client, // MQTT client handle
    const char *topic,               // Target topic
    const char *data,                // Message payload
    int len,                         // Length (0=auto)
    int qos,                         // QoS: 0=at most once
                                    //      1=at least once
                                    //      2=exactly once
    int retain                       // 0=do not retain
                                    // 1=Broker caches last message
);

```

7. Operation Flow

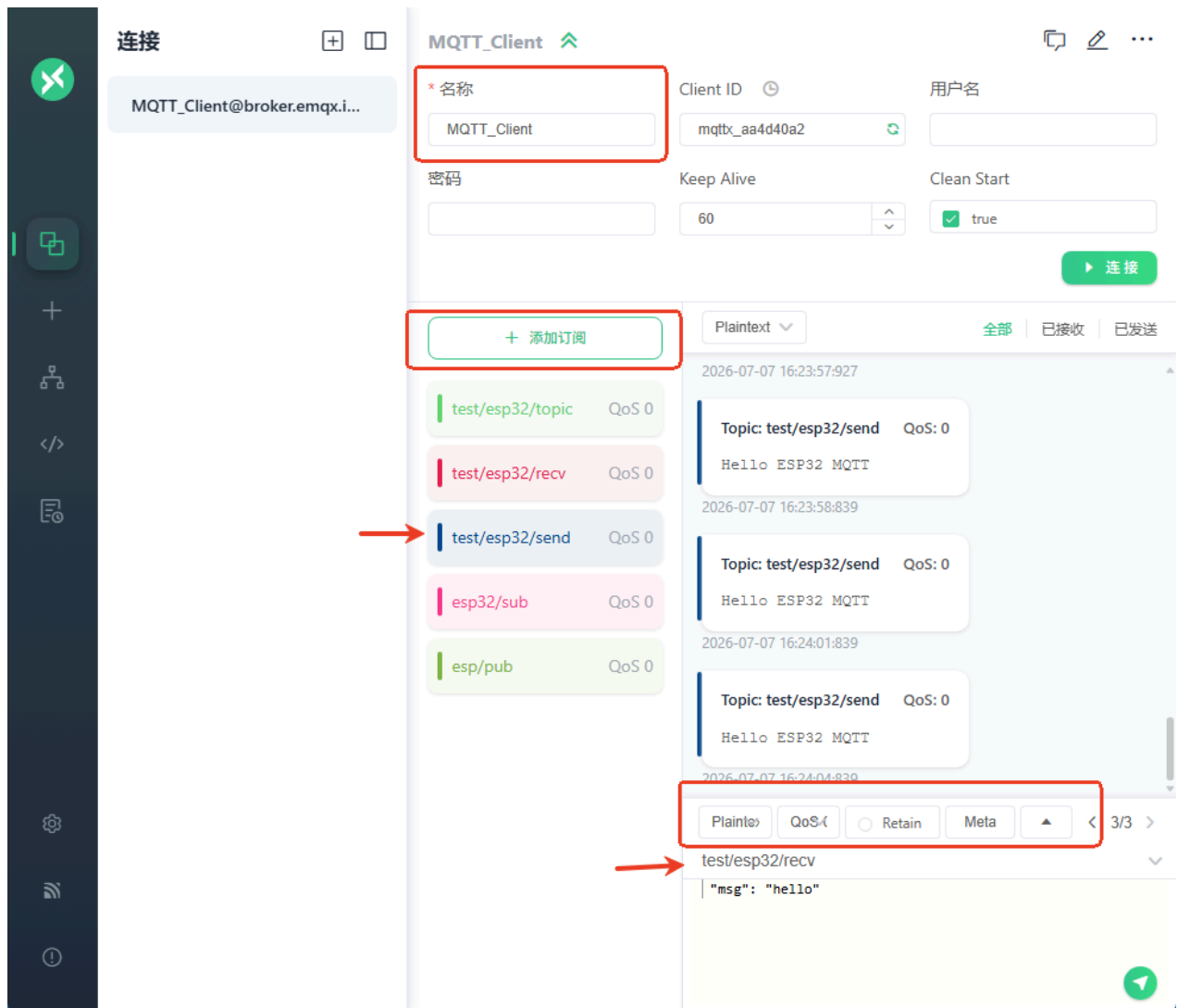
7.1 Build and Flash Flow

ESP-IDF Flash Flow: See `ESP-IDF → 1. Before Development → 3. Build and Flash`.

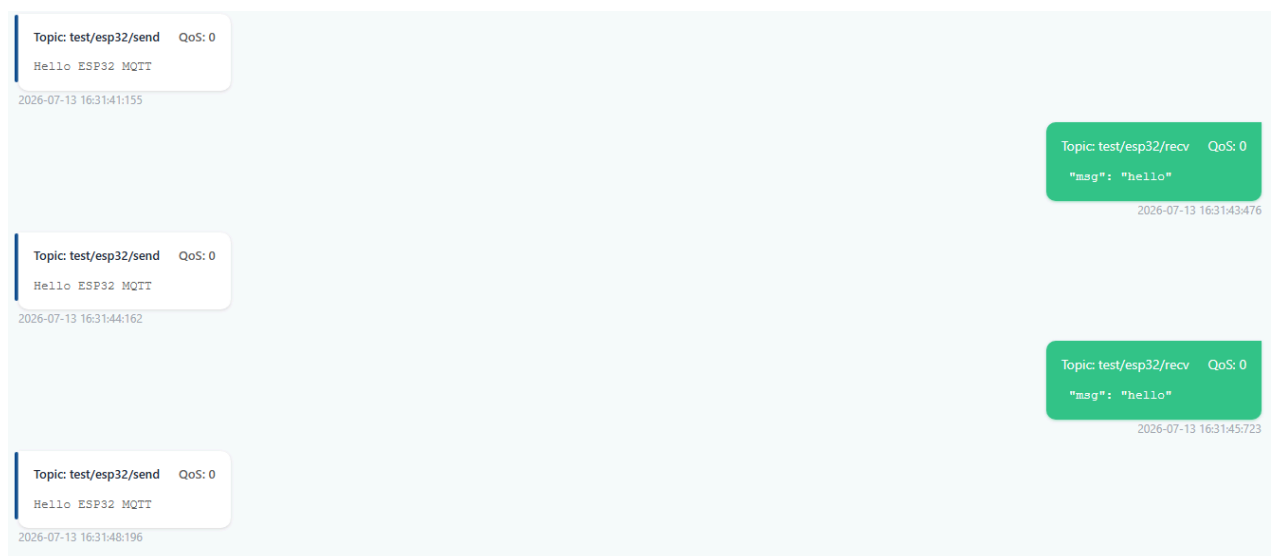
7.2 Testing Method

1. Ensure ESP32 WiFi can connect to external network (can ping `broker.emqx.io`).
2. Flash MQTT to ESP32, observe connection and sending logs on serial.
3. On PC, use **MQTTX** or **mobile MQTT debugging assistant**: For detailed usage, refer to: ([MQTT Client Tool Demo](#) | [EMQX 5.3 Documentation](#))





- Connect `broker.emqx.io:1883`
- Subscribe `test/esp32/send` → Receive messages published by ESP32 every 3 seconds
- Publish any message to `test/esp32/recv` → ESP32 serial prints received content



7.3 Normal Log

```
I (3856) MQTT-CLIENT: Got IP: 192.168.11.100
I (3858) MQTT-CLIENT: WiFi connected successfully!
I (4860) MQTT-CLIENT: MQTT Connected
I (4862) MQTT-CLIENT: Subscribed to receive topics: test/esp32/recv
I (4864) MQTT-CLIENT: Send message: Hello ESP32 MQTT
I (7864) MQTT-CLIENT: Send message: Hello ESP32 MQTT
```

```
I (1722) esp_netif_handlers: sta ip: 192.168.11.181, mask: 255.255.255.0, gw: 192.168.11.1
I (1722) MQTT-CLIENT: Got IP: 192.168.11.181
I (1722) MQTT-CLIENT: WiFi connected successfully!
I (1742) wifi:<ba-add>idx:1 (ifx:0, 80:8f:1d:20:9d:f7), tid:0, ssn:0, winSize:64
I (2642) MQTT-CLIENT: MQTT Connected
I (2642) MQTT-CLIENT: Subscribed to receive topics: test/esp32/recv
I (2642) MQTT-CLIENT: Send message: Hello ESP32 MQTT
I (5642) MQTT-CLIENT: Send message: Hello ESP32 MQTT
I (6232) MQTT-CLIENT: Receive remote messages: "msg": "hello"
I (8642) MQTT-CLIENT: Send message: Hello ESP32 MQTT
I (9522) MQTT-CLIENT: Receive remote messages: "msg": "hello"
I (11642) MQTT-CLIENT: Send message: Hello ESP32 MQTT
I (12472) MQTT-CLIENT: Receive remote messages: "msg": "hello"
I (13832) MQTT-CLIENT: Receive remote messages: "msg": "hello"
```

When PC MQTTX sends "msg": "hello" to test/esp32/recv:

```
I (9000) MQTT-CLIENT: Receive remote messages: {"msg": "hello"}
```

8. Common Problems

Symptom	Cause	Solution
<code>MQTT_EVENT_CONNECTED</code> never triggers	Broker address wrong or WiFi no external network	Confirm <code>BROKER_URI</code> correct; ping <code>broker.emqx.io</code> test external network
Compilation error <code>mqtt_client.h</code> not found	<code>espressif_mqtt</code> dependency not added	Add <code>espressif/mqtt: "*" in main/idf_component.yml</code>
Subscribe but receive no messages	Subscribe and publish topics inconsistent	Check <code>TOPIC_RECV</code> exactly matches PC publish topic
Publish successful but PC didn't receive	PC not subscribed or subscribed to different topic	Confirm PC has subscribed to <code>test/esp32/send</code>
Broker frequently disconnects and reconnects	Keep Alive interval too large	Set <code>.session.keepalive = 60</code> in <code>esp_mqtt_client_config_t</code>
MQTT message Chinese garbled	Terminal encoding issue	MQTT natively supports any binary data (including UTF-8), check display end encoding

Symptom	Cause	Solution
Need TLS/SSL encryption	<code>mqtt://</code> is plaintext transmission	Change to <code>mqtts://broker.emqx.io:8883</code> and configure certificate